MANUAL TÉCNICO – BACKEND

Introducción

El presente documento describe la arquitectura, funcionamiento y estructura del backend de la aplicación web desarrollada para la evaluación de catedráticos y cursos de la Facultad de Ingeniería.

El backend está construido como una **API REST** utilizando **Node.js**, permitiendo la comunicación con el frontend (React/Angular) y gestionando la persistencia de datos en una base de datos **MySQL**.

Arquitectura del Sistema

El sistema sigue una arquitectura **cliente-servidor**:

- **Cliente:** Aplicación web (React)
- **Servidor:** API REST en Node.js
- **Base de datos:** MySQL

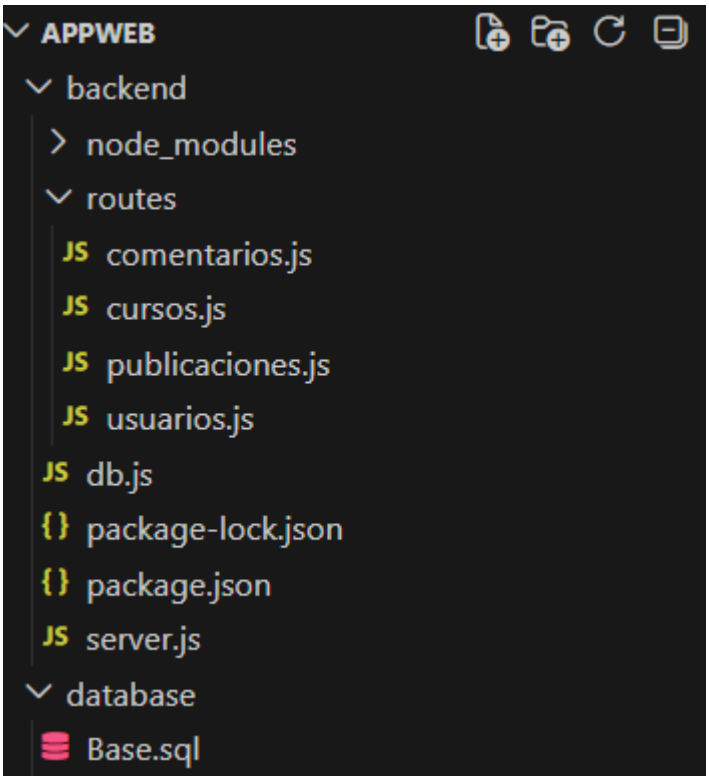
Flujo general:

1. El cliente envía una petición HTTP (GET/POST)
2. El servidor procesa la solicitud
3. Se consulta o modifica la base de datos
4. Se devuelve una respuesta en formato JSON

Tecnologías Utilizadas

- Node.js
- Express.js
- MySQL
- Cors
- Body-parser
- Postman (para pruebas de endpoints)

Estructura del Proyecto



- **db.js:** Configuración de conexión a MySQL
- **server.js:** Configuración del servidor y rutas
- **routes/**: Contiene los endpoints organizados por entidad

Conexión a la Base de Datos

El backend se conecta a MySQL mediante el paquete mysql2

db.js

backend > db.js > ...

```javascript
const mysql = require('mysql2')

const connection = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'Mercury/-68',
    database: 'web'
})

connection.connect(err => {
    if(err){
        console.log("Error de conexión", err)
    }else{
        console.log("Conectado a MySQL")
    }
})

module.exports = connection
```

Endpoints del Sistema

1) Usuarios

- Registro de usuario

POST http://localhost:3000/usuarios/registro

Docs Params Authorization Headers 8 Body S

none form-data x-www-form-urlencoded r

```
1 {
2   "registro_academico": "20237041",
3   "nombres": "Mina",
4   "apellidos": "Ashido",
5   "correo": "ins30/@mail.com",
6   "contraseña": "nit62"
7 }
```

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize

```
1 {
2     "mensaje": "Usuario registrado"
3 }
```

Permite registrar nuevos usuarios en la base de datos.

- Inicio de sesión

POST http://localhost:3000/usuarios/login

Docs Params Authorization Headers 8 Body

none form-data x-www-form-urlencoded

```
1 {
2   "correo": "dkam@gmail.com",
3   "contraseña": "6get8"
4 }
```

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize

```
1 {
2     "id_usuario": 1,
3     "registro_academico": 20210301,
4     "nombres": "Denki",
5     "apellidos": "Kaminari",
6     "correo": "dkam@gmail.com",
7     "contraseña": "6get8"
8 }
```

Valida las credenciales del usuario.

---

2) Publicaciones

- Obtener publicaciones

GET http://localhost:3000/publicaciones

Docs Params Authorization Headers 6 Body Scripts

none form-data x-www-form-urlencoded raw

```
1
```

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize


```
1  [
2      {
3          "id_publicacion": 1,
4          "id_usuario": 1,
5          "id_curso": 1,
6          "id_catedratico": 1,
7          "mensaje": "Trato de ser justo",
8          "fecha": "2026-03-17T22:34:19.000Z",
9          "nombres": "Denki"
10     }
11 ]
```


Retorna todas las publicaciones ordenadas por fecha descendente.

- Crear publicación

POST http://localhost:3000/publicaciones/crear

Docs Params Authorization Headers 8 Body Scripts

none form-data x-www-form-urlencoded raw

```
1 {
2   "id_usuario": 1,
3   "id_curso": 1,
4   "id_catedratico": null,
5   "mensaje": "No es facil, pero vale la pena"
6 }
```

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize

```
1 {
2     "mensaje": "Publicación creada"
3 }
```

Permite crear una publicación asociada a un curso o catedrático.

---

3) Comentarios

- Obtener comentarios

GET http://localhost:3000/comentarios/1

Docs Params Authorization Headers 6 Body Script

none form-data x-www-form-urlencoded raw

```
1
```

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize


```
1  [
2      {
3          "id_comentario": 1,
4          "id_publicacion": 1,
5          "id_usuario": 1,
6          "mensaje": "Que se muestre!!!!",
7          "fecha": "2026-03-17T22:34:19.000Z"
8      }
9  ]
```


Obtiene todos los comentarios asociados a una publicación.

- Crear comentario

POST http://localhost:3000/comentarios/crear

Docs Params Authorization Headers 8 Body

none form-data x-www-form-urlencoded

```
1 {
2   "id_publicacion": 1,
3   "id_usuario": 1,
4   "mensaje": "Funciona adecuado"
5 }
```

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize

```
1 {
2     "mensaje": "Comentario agregado"
3 }
```

Permite agregar comentarios a una publicación.

---

4) Cursos

- Obtener cursos

GET http://localhost:3000/cursos

Docs Params Authorization Headers 6 Body Sc

none form-data x-www-form-urlencoded

Body Cookies Headers 8 Test Results 1/1

{} JSON Preview Visualize


```
[
    {
        "id_curso": 1,
        "nombre_curso": "Programacion 1",
        "creditos": 5,
        "area": "Sistemas"
    }
]
```


Retorna todos los cursos disponibles.

Modelo de Base de Datos

El sistema utiliza las siguientes tablas:

- usuario
- catedratico
- curso
- publicacion
- comentario
- curso_aprobado

Las relaciones principales incluyen:

- Usuario → Publicaciones (1:N)
- Usuario → Comentarios (1:N)
- Publicación → Comentarios (1:N)
- Usuario ↔ Curso (N:M mediante curso_aprobado)

Manejo de Errores

El sistema maneja errores básicos:

- Error de conexión a base de datos
- Credenciales incorrectas
- Fallos en inserciones

Pruebas del Sistema

Los endpoints fueron probados utilizando **Postman**, verificando:

- Correcto funcionamiento de GET y POST
- Inserción de datos
- Respuestas JSON válidas

Conclusión

El backend desarrollado cumple con los requerimientos mínimos de la práctica, implementando una API REST funcional que permite la gestión de usuarios, publicaciones, comentarios y cursos, integrándose correctamente con una base de datos MySQL.